Lab06: Queue, RPN, Shunting-yard, and BFS

Objective(s):

- Implement and use a generic stack MyStack<T>.

- Implement and use a generic queue MyQueue<T> / MyQueueL_XXYYYY<T>.

- Complete an RPN evaluator using MyStack<Double>.

- Complete Shunting-yard conversion using an output queue and an operator stack.

- Apply queue-based BFS to the Largest Island challenge.

Overview

This lab connects the Stack and Queue lectures. MyStack<T> is used for RPN and operators. MyQueue<T> is used for output tokens in Shunting-yard and for BFS-style traversal.

All expressions in this lab are space-separated. For example:

```
( 4 + 2 ) / 3 * ( 8 - 5 )
```

Task 1: Implement MyStack<T>

Implement a generic stack backed by ArrayList<T>. Keep ArrayList private; do not extend ArrayList.

Required methods:

- void push(T data)
- T pop()
- T peek()
- int size()
- boolean isEmpty()
- String toString() // show top first

```
/* accompanied file should start here */
package Lab05.pack;


import java.util.ArrayList;


public class MyStack<T> {
    private ArrayList<T> items = new ArrayList<>();
```

```
    public void push(T data) {
        /* your code */
    }
    public T pop() {
        /* your code */
    }
    public T peek() {
        /* your code */
    }
    public int size() {
        /* your code */
    }
    public boolean isEmpty() {
        /* your code */
    }
    public String toString() {
        /* your code */
    }
}
```

Task 2: Implement MyQueue<T>

Implement a generic queue. In the current template this class may be named MyQueueL_XXYYYY<T>. It may wrap LinkedList<T> to keep this lab focused on queue behavior and Shunting-yard.

Required methods:

- void enqueue(T data)
- T dequeue()
- T peek()
- boolean isEmpty()
- Iterator<T> iterator()

```
/* accompanied file should start here */
package Lab06.pack;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;

public class MyQueueL_XXYYYY<T> implements Iterable<T> {
    private List<T> items = new LinkedList<>();

    public void enqueue(T d) {
        /* your code */
    }
    public T dequeue() {
        /* your code */
    }
    public T peek() {
        /* your code */
    }
    public boolean isEmpty() {
        return items.isEmpty();
    }
    public Iterator<T> iterator() {
        /* your code */
    }
}
```

Queue direction: enqueue adds at the back; dequeue removes from the front.

Task 3: Complete RPN Evaluation

Implement computeRPN(String rpn) in MyRPN.java. Use MyStack<Double>.

Algorithm:

- If the token is a number, push it.

- If the token is an operator, pop b, pop a, then push a op b.

- For subtraction and division, the order a op b matters.

- At the end, one value should remain on the stack.

```
public static double computeRPN(String rpn) {
    MyStack<Double> stack = new MyStack<>();
    StringTokenizer st = new StringTokenizer(rpn);
    while (st.hasMoreTokens()) {
        String t = st.nextToken();
        if (isNumeric(t)) {
            /* push number */
        } else {
            /* pop b, pop a, push a op b */
        }
    }
    return stack.pop();
}
```

Test case:

```
"8 5 - 4 2 + 3 / *"  ->  6.0
```

Task 4: Complete Shunting-yard

Implement infixToPostfix(String infixString) in MyShuntingYard.java. Use MyQueueL_XXYYYY<String> for output and MyStack<String> for operators.

Supported input: numbers, +, -, *, /, (,). Tokens are separated by spaces.

Rules:

- Number -> enqueue to output queue.

- '(' -> push to operator stack.

- ')' -> pop operators to output queue until '(' is found; discard '('.

- Operator -> while the top operator has higher or equal precedence, pop it to output queue; then push the current operator.

- After all tokens, pop remaining operators to output queue.

```
public static String infixToPostfix(String infixString) {
    MyQueueL_XXYYYY<String> queue = new MyQueueL_XXYYYY<>();
    MyStack<String> stack = new MyStack<>();

    StringTokenizer st = new StringTokenizer(infixString);
    while (st.hasMoreTokens()) {
        String t = st.nextToken();
        /* your code */
    }

    /* move remaining operators to queue */
    /* build and return postfix string from queue */
}
```

Demo:

```
String inFix = "( 4 + 2 ) / 3 * ( 8 - 5 )";
String postFix = MyShuntingYard.infixToPostfix(inFix);
System.out.println("postFix = " + postFix);
System.out.println("answer = " + MyRPN.computeRPN(postFix));

Expected postfix: 4 2 + 3 / 8 5 - *
Expected answer: 6.0
```

Task 5: Largest Island Challenge

Given a 2D grid of 0 and 1, return the size of the largest 4-directionally connected land region. Land is represented by 1. Water is represented by 0.

Use BFS. This challenge is intentionally lighter in explanation; connect it to the Queue lecture.

BFS idea:

- Scan every cell.

- When an unvisited land cell is found, start BFS from it.

- Use a queue of int[] cells.

- Mark a cell visited before enqueueing it.

- Count the connected land cells and keep the maximum.

```
public class Solution_XXYYYY {
    public int maxLandArea(int[][] grid) {
        int max = 0;
        /* your code */
        return max;
    }

    private int bfs(int[][] grid, boolean[][] visited, int startR, int startC) {
        /* your code */
    }
}
```

Test grid:

```
int[][] grid = {
    {1, 1, 0, 0},
    {1, 0, 0, 1},
    {0, 0, 1, 1},
    {1, 1, 0, 0}
};
Expected largest land area: 3
```

Submission

This pdf, MyStack_XXYYYY.java, MyQueueL_XXYYYY.java, MyRPN_XXYYYY.java and MyShuntning_XXYYYY.java and Lab06_XXYYYY.java

Due date: TBA